5. Perform setting up and Installing Hadoop in its two operating modes: pseudo distributed and fully distributed.

```
$ sudo apt update -y
$ sudo apt install openjdk-8-jdk -y
$ sudo wget https://archive.apache.org/dist/hadoop/core/hadoop-2.7.3/hadoop-2.7.3.tar.gz
$ tar -xvf hadoop-2.7.3.tar.gz
$ sudo mkdir /home/mca/Hadoop
$ chmod 775 -R  /home/mca/Hadoop
$ sudo mv ~/hadoop-2.7.3 /home/mca/Hadoop
$ sudo apt install openssh-server openssh-client -y
$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
$ chmod 0600 ~/.ssh/authorized_keys
$ ssh localhost
```

$ sudo nano ~/.bashrc

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/home/mca/Hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export PATH=$PATH:$HADOOP_HOME/bin
export PATH=$PATH:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```

$ source ~/.bashrc

$ sudo nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

$ sudo nano $HADOOP_HOME/etc/hadoop/core-site.xml

```
<property>
        <name>hadoop.tmp.dir</name>
        <value>/home/mca/tmpdata</value>
        <description>A base for other temporary directories.</description>
</property>
<property>
        <name>fs.default.name</name>
        <value>hdfs://localhost:54310</value>
        <description>The name of the default file system></description>
 </property>
```

$ sudo mv $HADOOP_HOME/etc/hadoop/mapred-site.xml.template $HADOOP_HOME/etc/hadoop/mapred-site.xml

$ sudo nano $HADOOP_HOME/etc/hadoop/mapred-site.xml

```
<property>
         <name>mapred.job.tracker</name>
         <value>localhost:54311</value>
</property>
```

$ sudo nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml

```
<property>
         <name>dfs.replication</name>
         <value>1</value>
</property>

<property>
        <name>dfs.namenode.name.dir</name>
        <value>/home/mca/dfsdata/namenode</value>
</property>

<property>
        <name>dfs.datanode.name.dir</name>
        <value>/home/mca/dfsdata/datanode</value>
</property>
```

Check Hadoop after Installing

In Browser: `http://localhost:50070`

```
$ hdfs namenode -format
$ start-all.sh
$ jps
```

6. Implement the following file management tasks in Hadoop: adding files and directories, Retrieving files and Deleting files

```
$ hdfs namenode -format

$ start-all.sh

$ jps

$ hdfs dfs -mkdir /testing

$ hdfs dfs -ls -R /

$ hdfs dfs -mkdir /testing2

$ hdfs dfs -mv /testing2 /testing

$ gedit sample.txt

$ hdfs dfs -put ~/sample.txt /testing

$ hdfs dfs -ls -R /

$ hdfs dfs -mv /testing/sample.txt /testing/testing2

$ hdfs dfs -cat /testing/testing2/sample.txt

$ ls

$ rm sample.txt

$ hdfs dfs -get /testing/testing2/sample.txt

$ hdfs dfs -rm /testing/testing2/sample.txt

$ hdfs dfs -rmdir /testing/testing2

$ hdfs dfs -touchz /testing/about.txt

$ hdfs dfs -rm /testing/about.txt

$ hdfs dfs -rmdir /testing
```

7. Performing a MapReduce Job for word search count (look for specific keywords in a file)

sample.txt

```
Hadoop is the Elephant King!
A yellow and elegant thing.
He never forgets Useful data.
A wonderful king is Hadoop.
Hadoop is an elegant fellow.
An elephant gentle and mellow.
Because, at his core, he is yellow.
A wonderful king is Hadoop.
```

mapper.py

```
#!/usr/bin/env python2
import sys
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print '%s\t%s' % (word, 1)
```

reducer.py

```
#!/usr/bin/env python2
from operator import itemgetter
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print '%s\t%s' % (current_word, current_count)
        current_count = count
        current_word = word
if current_word == word:
    print '%s\t%s' % (current_word, current_count)
```

```
$ sudo apt install python2
$ cat sample.txt | python2 mapper.py | sort -k1,1 | python2 reducer.py
$ hdfs namenode -format
$ start-all.sh
$ hdfs dfs -mkdir /testing
$ hdfs dfs -put ~/Desktop/sample.txt /testing
$ hadoop jar /home/mca/Hadoop/share/hadoop/tools/lib/hadoop-streaming-2.7.3.jar -input
/testing/sample.txt -output /testing/output -mapper ~/Desktop/mapper.py -reducer
~/Desktop/reducer.py

$ hdfs dfs -ls -R /
$ hdfs dfs -cat /testing/output/part-00000
```

8. Implement a Map Reduce program that processes a weather data set to: Find average, max and min temperature for each year in National Climate Data Centre data set.

Downloading Sample Weather DataSet

```
$ wget https://www.ncei.noaa.gov/pub/data/uscrn/products/daily01/2020/CRND0103-2020-
AK_Fairbanks_11_NE.txt

$ cp RND0103-2020-AK_Fairbanks_11_NE.txt weather.txt
```

First Open **Eclipse** -> then select **File -> New -> Java Project** ->Name it **MyProject** -> then select **use an execution environment** -> choose **JavaSE-1.8** then **next** -> **Finish**.

In this Project Create Java class with name **MyMaxMin** -> then click **Finish**

MyMaxMin.java

```
// importing Libraries
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;

public class MyMaxMin {
    public static class MaxTemperatureMapper extends Mapper<LongWritable, Text, Text, Text> {

    @Override
    public void map(LongWritable arg0, Text Value, Context context)throws IOException,
InterruptedException {
        String line = Value.toString();
              if (!(line.length() == 0)) {
                     String date = line.substring(6, 14);
                     float temp_Max = Float.parseFloat(line.substring(39, 45).trim());

                     float temp_Min = Float.parseFloat(line.substring(47, 53).trim());
                     if (temp_Max > 30.0) {

                     context.write(new Text("The Day is Hot Day :" + date),new
Text(String.valueOf(temp_Max)));
                     }
                     if (temp_Min < 15) {

                            context.write(new Text("The Day is Cold Day :" + date),new
Text(String.valueOf(temp_Min)));
                     }
              }
        }

    }
    public static class MaxTemperatureReducer extends Reducer<Text, Text, Text, Text>{
       public void reduce(Text Key, Iterator<Text> Values, Context context) throws IOException,
InterruptedException {
```

```
            String temperature = Values.next().toString();
            context.write(Key, new Text(temperature));
        }
    }
  public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = new Job(conf, "weather example");
        job.setJarByClass(MyMaxMin.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setMapperClass(MaxTemperatureMapper.class);
        job.setReducerClass(MaxTemperatureReducer.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        Path OutputPath = new Path(args[1]);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        OutputPath.getFileSystem(conf).delete(OutputPath);
        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }
}
```

Now we add these external jars to our **MyProject**. Right Click on **MyProject** -> then select **Build Path**-> Click on **Configure Build Path** and select **Add External jars….** and add jars from it's download location then click -> **Apply and Close**.

External Path jars Path

```
/home/mca/Hadoop/share/hadoop/common/hadoop-common-2.7.3.jar
/home/mca/Hadoop/share/hadoop/mapreduce/hadoop-mapreduce-client-core-2.7.3.jar
```

Now export the project as jar file. Right-click on **MyProject** choose **Export..** and go to **Java -> JAR file** click -> **Next** and choose your export destination then click -> **Next**.
choose Main Class as **MyMaxMin** by clicking -> **Browse** and then click -> **Finish** -> **Ok**.

Start Hadoop Daemons

```
$ hdfs namenode -format
$ start-all.sh
$ hdfs dfs -mkdir /testing
$ hdfs dfs -put ~/weather.txt /testing
$ hadoop jar ~/weather.jar /testing/weather.txt /output
$ hdfs dfs -cat /output/part-r-00000
```

Check in Browser : `http://localhost:50070`

9. Install and run Hive then use Hive to create, alter, and drop databases, tables and views

```
$ sudo apt update -y

$ wget https://downloads.apache.org/hive/hive-3.1.2/apache-hive-3.1.2-bin.tar.gz
$ tar xzf apache-hive-3.1.2-bin.tar.gz

$ sudo nano ~/.bashrc
```

#Add below lines in this file

```
export HIVE_HOME=/home/mca/apache-hive-3.1.2-bin
export PATH=$PATH:$HIVE_HOME/bin
```

```
$ sudo nano $HIVE_HOME/bin/hive-config.sh
```

#Add below lines in this file

```
export HADOOP_HOME=/home/mca/Hadoop
```

Create Hive directories in HDFS
==========

```
$ hdfs dfs -mkdir /tmp
$ hdfs dfs -chmod g+w /tmp
$ hdfs dfs -mkdir -p /user/hive/warehouse
$ hdfs dfs -chmod g+w /user/hive/warehouse
```

Initialize Derby and hive
==========

```
$ schematool -initSchema -dbType derby
```

Launching Hive
==========

```
$ hive
```

Create Database & Table , Alter , Insert, Read, Load , View ,Drop

OPEN TERMINAL -> `gedit student_details.txt`

Type the Details:

```
10001,Aazim,99,MCA
10002,Antony,98,MBA
10003,Akash,35,MCA
10004,Manoj,45,MBA
10005,Prem,89,ME
```

Save the file

```
hive> SHOW DATABASES;

hive> CREATE DATABASE IF NOT EXISTS Student_Details;

hive> SHOW DATABASES;

hive> USE Student_Details;

hive> CREATE TABLE IF NOT EXISTS Student(Stud_id int, Stud_Name String, Stud_Mark int,
Stud_Dept String) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n' STORED
AS TEXTFILE ;

hive> SHOW TABLES;

hive> DESCRIBE  Student;

hive> ALTER TABLE Student RENAME TO Stud;

hive> SHOW DATABASES;

hive> LOAD DATA LOCAL INPATH '/home/mca/student_details.txt' OVERWRITE INTO TABLE Stud;

hive> SELECT * FROM Stud;

hive> SELECT * FROM Stud WHERE stud_mark<50;

hive> CREATE VIEW PASS AS SELECT * FROM Stud WHERE stud_mark<50;

hive> SELECT * FROM PASS;

hive> DROP TABLE Stud;

hive> SHOW TABLES;

hive> DROP DATABASE Student_Details CASCADE;

hive> SHOW DATABASES;
```

10. Install, deploy & configure Apache Spark cluster. Run Apache Spark applications using Scala

```
$ sudo apt update

$ sudo apt-get install default-jdk -y

$ java  -version

$ wget https://dlcdn.apache.org/spark/spark-3.2.4/spark-3.2.4-bin-hadoop2.7.tgz

$ tar -xvf spark-3.2.4-bin-hadoop2.7.tgz

$ chmod -R 775  spark-3.2.4-bin-hadoop2.7

$ sudo nano ~/.bashrc

      SPARK_HOME=/home/mca/spark-3.2.4-bin-hadoop2.7
      export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin



$ source ~/.bashrc

$ spark-shell

      scala> val nums = Array(1,2,3,5,6)

      scala> nums.foreach(println)

      scala> val data = sc.parallelize(nums)

      scala> data.foreach(println)
```

Creating a cluster for Apache Spark

```
$ cd $SPARK_HOME/conf/

$ cp spark-env.sh.template spark-env.sh

$ sudo nano spark-env.sh

      SPARK_MASTER_HOST='192.168.0.102'

$ cd $SPARK_HOME/sbin/
```

Master node run command

```
      $ ./start-master.sh
```

Slave node run command

```
      $ ./start-slave.sh spark://192.168.0.102:7077
```

Check in Browser : `http://192.168.0.102:7077`